

OS CPU SCHEDULING

CPU SCHEDULING

2.5 Processor Scheduling algorithms

- The goal of a scheduling algorithm is to select the most appropriate process in the ready queue.
- For simplicity, we will assume that
 - we have a **single I/O server** and a **single device queue**,
 - device queue always implemented with **FCFS (FIFO)** method.
- We also will **neglect** the **switching time** between processes (*context switching*).

2.3.1 First-Come-First-Served (FCFS)

- In this algorithm, the process to be selected is the process which requests the processor first.
- This is the process whose PCB is at the head of the ready queue.
- Contrary to its simplicity, its performance may often be poor compared to other algorithms.

2.3.1 First-Come-First-Served (FCFS)

- FCFS may cause processes with short processor bursts to wait for a long time.
- If one process with a long processor burst gets the processor, all the others will wait for it to release it and the ready queue will be crowded.
- This is called the **convoy** *effect*.

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	8
P3	3	6	16



Process Wait Time : **Service Time - Arrival**
Time

P0 $0 - 0 = 0$

P1 $5 - 1 = 4$

P2 $8 - 2 = 6$

P3 $16 - 3 = 13$

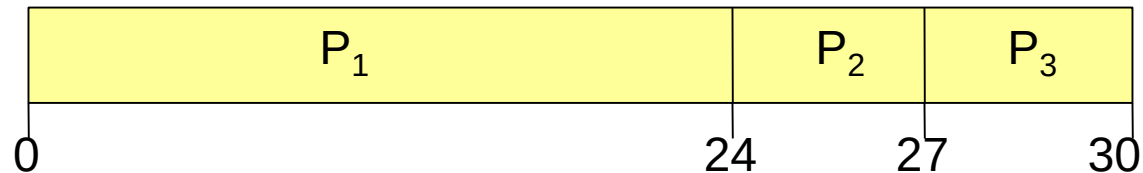
Average Wait Time: $(0+4+6+13) / 4 = 5.55$

FCFS

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- With FCFS, the process that requests the CPU first is allocated the CPU first
- Case #1: Suppose that the processes arrive in the order: P_1 , P_2 , P_3

The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27) / 3 = 17$
- Average turn-around time: $(24 + 27 + 30) / 3 = 27$

SJF

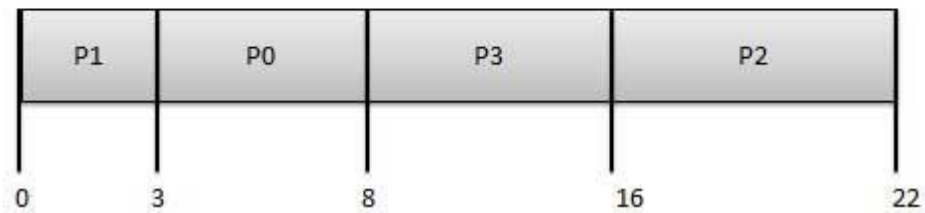
- The SJF algorithm associates with each process the length of its next CPU burst
- When the CPU becomes available, it is assigned to the process that has the smallest next CPU burst (in the case of matching bursts, FCFS is used)
- Two schemes:
 - **Nonpreemptive** – once the CPU is given to the process, it cannot be preempted until it completes its CPU burst
 - **Preemptive** – if a new process arrives with a CPU burst length less than the remaining time of the current executing process, preempt. This scheme is known as the Shortest-Remaining-Time-First (SRTF)
- Best approach to minimize waiting time.
- Impossible to implement
- Processer should know in advance how much time process will take.

- | Process | Arrival Time | Burst Time |
|-----------|--------------|------------|
| <i>P1</i> | 0.0 | 7 |
| <i>P2</i> | 2.0 | 4 |
| <i>P3</i> | 4.0 | 1 |
| <i>P4</i> | 5.0 | 4 |



- **Average waiting time = $[0 + (8-2) + (7-4) + (12-5)] / 4 = 4$**

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	3
P2	2	8	8
P3	3	6	16



Process

Wait Time : Service Time - Arrival Time

P0

$$3 - 0 = 3$$

P1

$$0 - 0 = 0$$

P2

$$16 - 2 = 14$$

P3

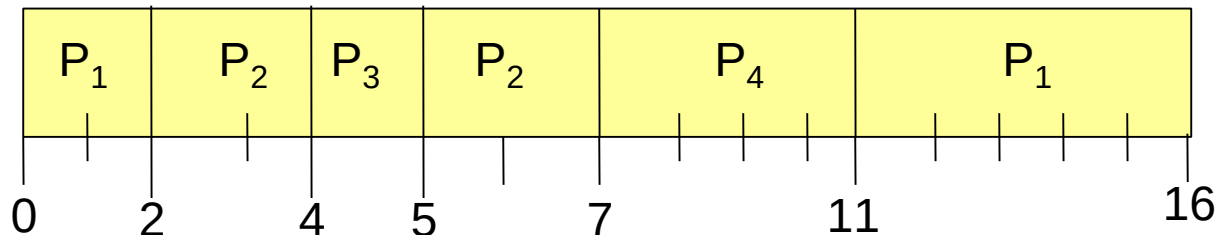
$$8 - 3 = 5$$

Average Wait Time: $(3+0+14+5) / 4 = 5.50$

Preemptive SJF (Shortest-remaining-time-first)

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive, varied arrival times)



- Average waiting time

$$= ([(0 - 0) + (11 - 2)] + [(2 - 2) + (5 - 4)] + (4 - 4) + (7 - 5)) / 4$$

$$= (9 + 1 + 0 + 2) / 4$$

$$= 3$$
- Average turn-around time = $(16 + 7 + 5 + 11) / 4 = 9.75$

Priority Scheduling Algorithm

- The SJF algorithm is a special case of the general priority scheduling algorithm
- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer = highest priority)
- Priority scheduling can be either preemptive or non-preemptive
 - A **preemptive** approach will preempt the CPU if the priority of the newly-arrived process is higher than the priority of the currently running process
 - A **non-preemptive** approach will simply put the new process (with the highest priority) at the head of the ready queue
- SJF is a priority scheduling algorithm where priority is the predicted next CPU burst time
- The main problem with priority scheduling is **starvation**, that is, low priority processes may never execute
- A solution is **aging**; as time progresses, the priority of a process in the ready queue is increased

- Example

•	Process	Burst time	Arrival	Priority
•	<i>P1</i>	<i>10</i>	<i>0.0</i>	<i>3</i>
•	<i>P2</i>	<i>1</i>	<i>0.0</i>	<i>1</i>
•	<i>P3</i>	<i>2</i>	<i>0.0</i>	<i>4</i>
•	<i>P4</i>	<i>1</i>	<i>0.0</i>	<i>5</i>
•	<i>P5</i>	<i>5</i>	<i>0.0</i>	<i>2</i>

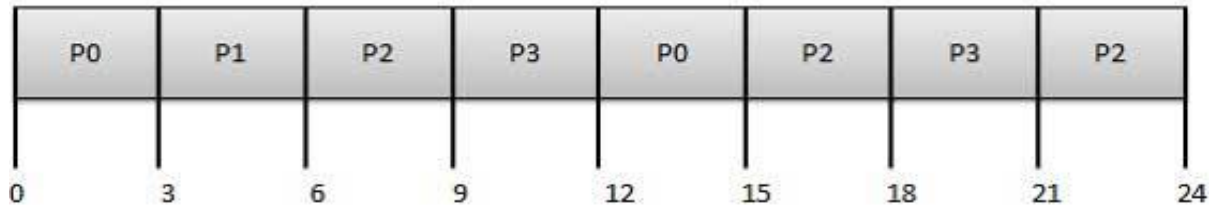
- Average waiting time = $(0 + 1 + 6 + 16 + 18) / 5 = 8.2$

Round Robbin

- In the round robin algorithm, each process gets a small unit of CPU time (a *time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Performance of the round robin algorithm
 - q large $\Rightarrow$ FCFS
 - q small $\Rightarrow q$ must be greater than the context switch time; otherwise, the overhead is too high
- One rule of thumb is that 80% of the CPU bursts should be shorter than the time quantum

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

Quantum = 3



Process

Wait Time : Service Time - Arrival Time

P0

$$(0-0) + (12-3) = 9$$

P1

$$(3-1) = 2$$

P2

$$6-2) + (15-9) = 10$$

P3

$$(9-3) + (18-12) = 12$$

Average Wait Time: $(9+2+10+12) / 4 = 8.25$